

Étude d'une stratégie de résolution du Jeu de Nim et conception d'un algorithme pour ledit jeu basé sur l'apprentissage automatique

EGBAKOU Kodzovi David

SCEI 18709

- 1 Introduction
- 2 Analyse théorique
- 3 Travail Pratique
- 4 Résultats

- 1 Introduction
 - Problématique
- 2 Analyse théorique
- 3 Travail Pratique
- 4 Résultats

Introduction

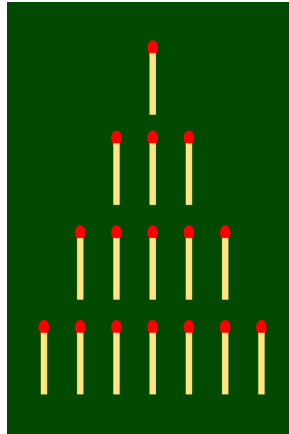


Figure – Présentation du jeu de Nim

Problématique

Comment modéliser un algorithme assurant la victoire au jeu du Nim ?

- 1 Introduction
- 2 Analyse théorique
 - Solution binaire
 - Approche par les graphes
- 3 Travail Pratique
- 4 Résultats

La solution de résolution binaire se base sur la décomposition en binaire puis à partir d'un calcul à l'aide d'une *somme digitale* on obtient une solution interprétable.

Cette méthode se déroule en 4 étapes :

- Formation de la matrice binaire associée à la configuration
- Calcul de la somme digitale de chaque colonne
- Vérification de la nature de la position du joueur

Matrice associée

Soit une matrice M de taille (n, m) . M est la matrice associée à la configuration du jeu à un instant donné

On a :

$$M = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix} \quad (1)$$

tel que $\forall i, j \in \{1, 2, \dots, n\}, a_{ij} \in \{0, 1\}$

Somme digitale

Définition : Somme digitale

La somme digitale est un opérateur défini sur l'ensemble des entiers. Elle consiste en une somme modulo 2.

Elle sera notée dans la suite par $\oplus$

Soit M la matrice associée à une position au jeu de Nim :
Notons Sd_j la somme digitale de la $j^{\text{ème}}$ colonne. Alors on a

$$\forall j < m+1, Sd_j \equiv \left(\sum_{i=1}^n a_{ij} \right) \pmod{2}$$

Donc, la somme digitale appliquée à la matrice M est

$$a_{11}a_{12}\dots a_{1m} \oplus a_{21}a_{22}\dots a_{2m} \oplus \dots \oplus a_{n1}a_{n2}\dots a_{nm} = Sd_1 Sd_2 \dots Sd_m$$

Somme digitale

Propriétés de la somme digitale :

- Associativité :

$$(a_0 a_1 \dots a_n \oplus b_0 b_1 \dots b_n) \oplus c_0 c_1 \dots c_n = a_0 a_1 \dots a_n \oplus (b_0 b_1 \dots b_n \oplus c_0 c_1 \dots c_n)$$

- Commutativité : $a_0 a_1 \dots a_n \oplus b_0 b_1 \dots b_n = b_0 b_1 \dots b_n \oplus a_0 a_1 \dots a_n$

- Existence d'un élément neutre : $a_0 a_1 \dots a_n \oplus 0 = a_0 a_1 \dots a_n$

- Existence d'un opposé : $\forall a_0 a_1 \dots a_n, \exists b_0 b_1 \dots b_n \in \mathbb{N}, a_0 a_1 \dots a_n \oplus b_0 b_1 \dots b_n = 0$

Théorème de Bouton

Si la position dans laquelle se trouve un joueur possède une somme digitale non nulle, alors il se trouve en position gagnante ; sinon, ce sera son adversaire qui sera en position gagnante.

Le théorème de Bouton repose sur deux lemmes très important qui permettent de donner un sens à la nature de la position

Soit x_i le nombre d'allumettes de la $i^{\text{ème}}$ ligne avec i un entier compris entre 1 et n ; et $y_1, y_2, \dots, y_n$ les lignes après qu'un joueur est joué.

Notons $Sd_b = x_1 \oplus x_2 \oplus \dots \oplus x_n$ et $Sd_f = y_1 \oplus y_2 \oplus \dots \oplus y_n$

Lemme 1

Si un joueur se trouve en position perdante, alors tous ses coups mèneront son adversaire en position gagnante

Autrement si $Sd_b = 0$ alors $Sd_f \neq 0$

Lemme 2

Si le joueur se trouve en position gagnante, il existe au moins un coup qui nous permet de mettre notre adversaire dans une position perdante.

C'est-à-dire, si $Sd_b \neq 0$ alors il existe un mouvement tel que $Sd_f = 0$

Approche par les graphes

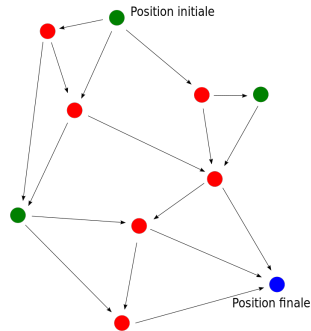


Figure – Représentation graphique du jeu de Nim

Approche par les graphes

Le jeu de Nim admet une représentation au travers d'un graphe orienté sans circuit.
Chaque sommet désigne une position possible au cours de la partie
Désignons par x la position actuelle, soit y une image de x et $\Phi(x)$ l'ensemble des images de x

Noyau d'un graphe

Le noyau K d'un graphe est l'ensemble des sommets qui vérifient :

- $x \in K \implies \Phi(x) \cap K = \emptyset$
- $x \notin K \implies \Phi(x) \cap K \neq \emptyset$

D'après le Lemme 2, K désigne l'ensemble des positions perdantes dans une partie

1 Introduction

2 Analyse théorique

3 Travail Pratique

- Définition de l'environnement de Jeu
- Définition de l'agent optimal
- Apprentissage par Q-learning

4 Résultats

Définition de la class NimEnv en utilisant un certain nombre de fonction

- `__init__(self, seed=None) → None`
- `check_valid(self, action) → bool`
- `step(self, action) → (self.heaps, self.end, self.winner)`
- `observe(self) → (self.heaps, self.end, self.winner)`
- `reward(self, player=0) → float`
- `reset(self, seed=None) → self.heaps.copy()`
- `render(self, simple=False) → None`

Données : Piles

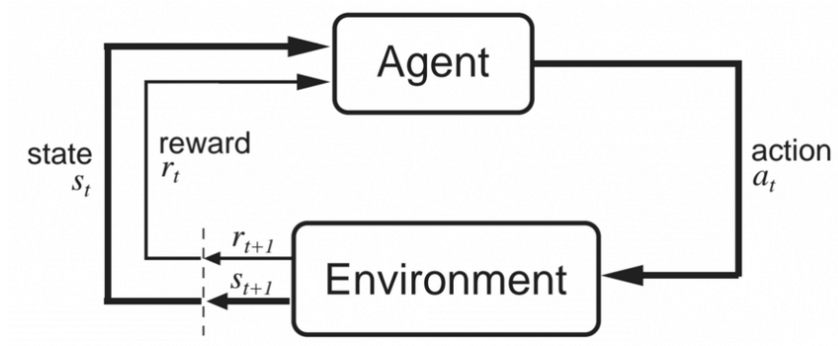
Résultat : un entier représentant le résultat de la somme digitale

- 1 Initialiser $nim = 0$;
- 2 **pour** *chaque éléments i de la Piles* **faire**
- 3 $nim \leftarrow nim \text{ XOR } i$;
- 4 **fin**
- 5 Retourne la valeur finale de nim

Algorithme 1 : Algorithme somme digitale

Le Q-learning est un algorithme d'apprentissage renforcée se basant sur trois étapes centrales à savoir :

- **Connaître** : définir une fonction action-valeur Q
- **Renforcement des connaissances** : Mettre à jour la fonction Q
- **Agir** : Adopter une stratégie d'action



Données : A, S, α : taux d'apprentissage, γ : facteur d'actualisation, $\epsilon \in \mathbb{R}_+^*$

Résultat : Une Q -table contenant les Q -valeurs d'états et d'actions de l'agent

```
1 Initialiser  $Q(s, a)$  de façon aléatoire pour tout  $(s, a) \in A \times S$ ,  $i = 1$  ;  
2 pour chaque épisode faire  
3   Initialiser  $s$ ;  
4   pour chaque étape  $t$  de l'épisode faire  
5     Choisir l'action  $a$  en utilisant l'algorithme  $\epsilon$ -greedy;  
6     Effectuer l'action  $a$ , et recevoir la récompense  $r$ , et observer le nouvel état  
        $s'$ ;  
7      $Q_{t+1}(s, a) \leftarrow Q_t(s, a) + \alpha[r + \gamma \max_{a' \in A} Q_t(s', a') - Q_t(s, a)]$ ,  $s \leftarrow s'$   
8   fin  
9 fin
```

Algorithme 2 : Algorithme Q-learning

Données : Q -table, $s \in S, \epsilon \in \mathbb{R}_+^*$

Résultat : Action choisie a

- 1 $\eta \leftarrow$ un nombre aléatoire dans $[0, 1]$;
- 2 **si** $\eta < \epsilon$ **alors**
- 3 | $a \leftarrow$ action aléatoire dans A ;
- 4 **sinon**
- 5 | $a \leftarrow \max_{a' \in A} Q(s, a')$;
- 6 **fin**
- 7 Retourne l'action a sélectionnée;

Algorithme 3 : Algorithme ϵ -greedy

1 Introduction

2 Analyse théorique

3 Travail Pratique

4 Résultats

- Indicateur de performance
- Résultats

La performance est définie par :

- Mean reward
- M_{opt} tel que :

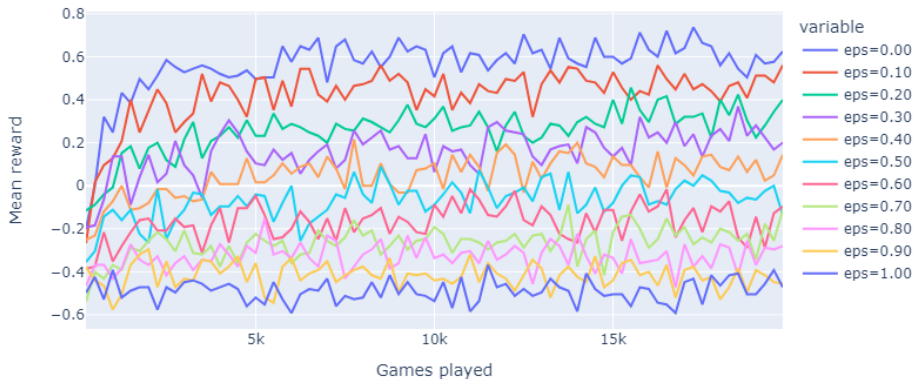
$$M_{opt} = \frac{N_{win} - N_{lose}}{N}$$

avec $N=500$

- M_{rand}

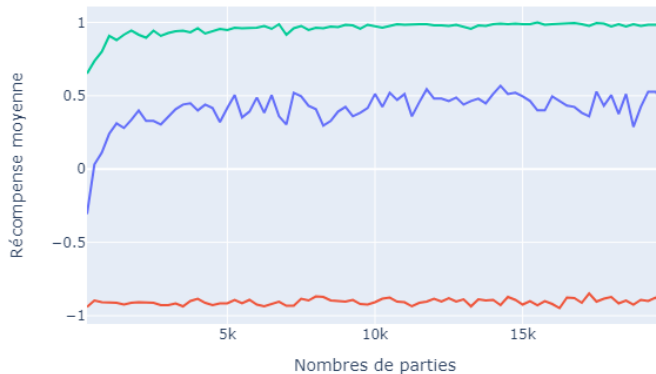
Résultats

Training and testing with self-learning, using different eps values



Résultats

Récompense moyenne au fil du temps de l'agent RL avec $\epsilon=0.1$

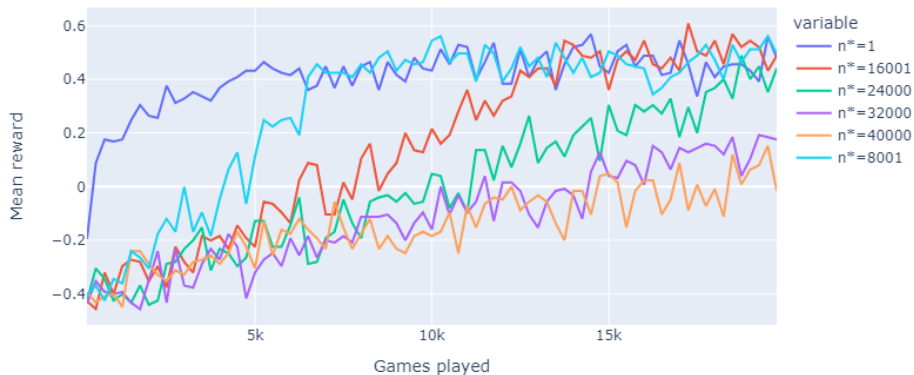


Pour rendre l'agent plus performant on fait décroître progressivement ϵ , le taux d'exploration Ainsi,

$$\epsilon(n) = \max\{\epsilon_{\min}, \epsilon_{\max} \left(1 - \frac{n}{n^*}\right)\}.$$

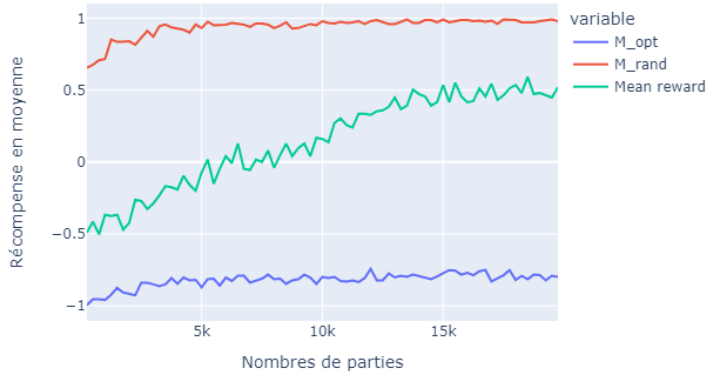
Résultats

Entraînement avec epsilon décroissant. Comparaison de différentes valeurs de n^*



Résultats

Entraînement de l'agent en faisant décroître epsilon avec $n^*=16000$



Conclusion

MERCI POUR VOTRE ATTENTION

EGBAKOU Kodzovi David

SCEI 18709

Considérons sd_b et sd_f défini plutôt et admettons que le joueur prenne une/des allumette(s) dans la ligne k : $\forall i \neq k, x_i = y_i$ tandis que $x_k \neq y_k$. En appliquant les propriétés de $\oplus$ (associativité, commutativité et que $x \oplus x = 0$), nous obtenons :

$$\begin{aligned}sd_f &= 0 \oplus sd_f \\&= sd_b \oplus sd_b \oplus sd_f \\&= sd_b \oplus (x_1 \oplus x_2 \oplus \dots \oplus x_n \oplus y_1 \oplus y_2 \oplus \dots \oplus y_n) \\&= sd_b \oplus (x_1 \oplus y_1) \oplus (x_2 \oplus y_2) \oplus \dots \oplus (x_k \oplus y_k) \oplus \dots \oplus (x_n \oplus y_n) \\&= sd_b \oplus 0 \oplus \dots \oplus (x_k \oplus y_k) \oplus \dots \oplus 0 \\&= sd_b \oplus x_k \oplus y_k \quad (1)\end{aligned}$$

Par récurrence à partir des 2 lemmes on obtient le théorème de Bouton

Annexe 1 : Démonstration

Démonstration Lemme 1 :

Comme $sd_b = 0$, alors $sd_f = x_k \oplus y_k$. Si le joueur modifie le nombre d'allumettes dans la ligne k et étant donné que $x_k \neq y_k$, alors $sd_f \neq 0$.

Annexe 1 : Démonstration

Démonstration du Lemme 2 :

Soit d la position du premier chiffre non nul de sd_b en commençant par la gauche. Prenons k tel que le d^{ieme} bit de x_k est également non nul (k existe sinon le d^{ieme} bit de sd_b vaudrait 0).

Posons $y_k = sd_b \oplus x_k$, alors $y_k \neq x_k$.

Tous les bits se trouvant à gauche de d sont semblables dans x_k et y_k . Le d^{ieme} bit qui est un 1 devient un 0 en faisant diminuer la valeur de 2^d .

Peu importe quels sont les changements apportés sur les bits à droite de d , ils ne créeront en aucun cas un changement supérieur à 2^{d-1} . Celui qui joue est en mesure de prendre $x_k - y_k$ allumettes de la ligne k et donc :

$$\begin{aligned}sd_f &= sd_b \oplus x_k \oplus y_k \\&= sd_b \oplus x_k \oplus (sd_b \oplus x_k) \\&= 0\end{aligned}$$

Annexe 2 : Code python

```
import random
import numpy as np

class NimEnv:
    def __init__(self, seed=None):
        self.n_heap = 3
        self.n_agents = 2
        self.current_player = 0
        self.winner = None
        self.end = False
        self.num_step = 0

        if seed is not None:
            random.seed(seed)
        self.heaps = random.sample(range(1, 8), 3)
        self.heap_avail = [True, True, True]
        self.heap_keys = ["1", "2", "3"]
        self.winner = None

    def check_valid(self, action):
        h, n = map(int, action)
        if not self.heap_avail[h - 1]:
            return False
        if n < 1:
            return False
        if n > self.heaps[h - 1]:
            return False
        return True

    def step(self, action):
        """
        step method takin an action as input

        Parameters
        -----
        action : list(int)
            action[0] = 1, 2, 3 is the selected heap to take from
            action[1] is the number of objects to take from the heap

        Returns
        -----
        getObservation()
            State space (printable).
        reward : tuple
            (0,0) when not in final state, +1 for winner and -1 for loser
            otherwise.
        done : bool
            is the game finished.
        dict
            dunno.
        """
```

Annexe 2 : Code python

```
# extracting integer values h: heap id, n: nb objects to take
h, n = map(int, action)

assert self.heap_avail[h - 1], "The selected heap is already empty"
assert n >= 1, "You must take at least 1 object from the heap"
assert (
    n <= self.heaps[h - 1]
), "You cannot take more objects than there are in the heap"

self.heaps[h - 1] -= n # core of the action

if self.heaps[h - 1] == 0:
    self.heap_avail[h - 1] = False

reward = (0, 0)
done = False
if self.heap_avail.count(True) == 0:
    done = True
    self.winner = self.current_player

self.end = done
# update
self.num_step += 1
self.current_player = 0 if self.num_step % 2 == 0 else 1
next_heaps = self.heaps[:]
return self.heaps, self.end, self.winner

def observe(self):
    return self.heaps, self.end, self.winner

def reward(self, player=0):
    if self.end:
        if self.winner is None:
            return 0
        else:
            return 1 if player == self.winner else -1
    else:
        return 0

def reset(self, seed=None):
    if seed is not None:
        random.seed(seed)
    self.heaps = random.sample(range(1, 8), 3)
    self.heap_avail = [True, True, True]
    self.current_player = 0
    self.winner = None
    self.end = False
    self.num_step = 0
    return self.heaps.copy()
```

Annexe 2 : Code python

```
def render(self, simple=False):  
    if simple:  
        print(self.heaps)  
    else:  
        print(u"\u2500" * 35)  
        for i in range(len(self.heaps)):  
            print(  
                "Heap {}: {:15s} \t {}".format(  
                    self.heap_keys[i], "|" * self.heaps[i], self.heaps[i]  
                )  
            )  
        print(u"\u2500" * 35)
```

Annexe 2 : code python

```
class OptimalPlayer:

    def __init__(self, player=0, epsilon=0.2):
        self.epsilon = epsilon
        self.player = player # 0 or 1

    def set_player(self, player=0, j=-1):
        self.player = player
        if j != -1:
            self.player = 0 if j % 2 == 0 else 1

    def randomMove(self, heaps):
        # the indexes of the heaps available are given by
        heaps_avail = [i for i in range(len(heaps)) if heaps[i] > 0]
        chosen_heap = random.choice(heaps_avail)
        n_obj = random.choice(range(1, heaps[chosen_heap] + 1))
        move = [chosen_heap + 1, n_obj]

        return move

    def compute_nim_sum(self, heaps):
        nim = 0
        for i in heaps:
            nim = nim ^ i
        return nim

    def act(self, heaps, **kwargs):
        if random.random() < self.epsilon:
            return self.randomMove(heaps)

        else:
            nim_sum = self.compute_nim_sum(heaps)
            if nim_sum == 0:
                # You will lose :(
                count_non_0 = sum(x > 0 for x in heaps)
                if count_non_0 == 0:
                    # Game is already finished, return a dumb move
                    move = [-1, -1]
                else:
                    # Take any possible move
                    move = [heaps.index(max(heaps)) + 1, 1]
            return move

        # Calc which move to make
        for index, heap in enumerate(heaps):
            target_size = heap ^ nim_sum
            if target_size < heap:
```

Annexe 2 : code python

```
        amount_to_remove = heap - target_size
        move = [index + 1, amount_to_remove]
        return move

def act_q(self, heaps, q_table_row, greedy=False):
    if not greedy and random.random() < self.epsilon:
        move = self.randomMove(heaps)
        # converting move (list of 2 int) in the corresponding action (q_table_row's
cell)
        if move[0] == 1:
            q_value = q_table_row[move[1]-1]
            action = move[1]-1
        if move[0] == 2:
            q_value = q_table_row[heaps[0] + move[1]-1]
            action = heaps[0] + move[1]-1
        if move[0] == 3:
            q_value = q_table_row[heaps[0] + heaps[1] + move[1]-1]
            action = heaps[0] + heaps[1] + move[1]-1
        else:
            # taking the highest q-value and saving the corresponding cell's index in
action, then
            # converting the action to the move format (list of 2 int)
            q_value = np.max(q_table_row)
            action = np.argmax(q_table_row)
            if action <= (heaps[0]-1):
                move = [1, action + 1]
            if action > (heaps[0]-1) and action <= (heaps[0] + heaps[1] - 1):
                move = [2, action - heaps[0] + 1]
            if action > (heaps[0] + heaps[1] - 1):
                move = [3, action - heaps[1] - heaps[0] + 1]
            return move, q_value, action
```

Annexe 2 : code python

```
def heaps_to_idx(heaps):  
    idx=0  
    for i in range(NUM_HEAPS):  
        idx += heaps[i]*((MAX_STICKS + 1)**(NUM_HEAPS-i-1))  
    return idx  
  
def idx_to_heaps(idx):  
    idx_copy=idx  
    heaps=[]  
    for i in range(NUM_HEAPS):  
        heaps.append(int(idx_copy//((MAX_STICKS + 1)**(NUM_HEAPS-i-1))))  
        idx_copy = idx_copy - heaps[i]*((MAX_STICKS + 1)**(NUM_HEAPS-i-1))  
    return heaps  
  
def create_q_table(num_heaps, max_sticks, initial_value=0.0):  
    q_table=[]  
    for i in range((max_sticks+1)**num_heaps):  
        if i==0:  
            q_table.append(np.full(shape=[1,1], fill_value=initial_value,dtype='float64'))  
            continue  
        else:  
            heaps = idx_to_heaps(i)  
            num_actions=0  
            for ii in range(num_heaps):  
                num_actions += heaps[ii]  
            q_table.append(np.full(shape=[num_actions,1], fill_value=initial_value,dtype='float64'))
```

Annexe 2 : code python

```
def play_game(env, q_table, player_q, player_adv, eta, gamma, test= False, self_learning= False):  
  
    # resetting the variables and the NimEnv  
    seed = random.uniform(0,9999)  
    env.reset(seed)  
    heaps, _, _ = env.observe()  
    reward = 0 # stays at 0 until game is finished  
    while not env.end:  
        if env.current_player == player_q.player: # player 1 : learning agent  
            # q-learning agent 1 choosing next action using q-values : making the move  
            state_idx = heaps_to_idx(heaps)  
            move, q_value, action = player_q.act_q(heaps, q_table[state_idx], greedy= False) # if greedy = False, it means eps-greedy  
            heaps, end, winner = env.step(move)  
            if self_learning and not test:  
                # q-learning agent 2 choosing next potential greedy-action using q-values, without actually taking it, then updating the q-table  
                next_state_idx_adv = heaps_to_idx(heaps)  
                _, q_value_next_adv, _ = player_adv.act_q(heaps, q_table[next_state_idx_adv], greedy= True)  
                if not end and env.num_step>1:  
                    q_table[state_idx][action] += eta * (reward + gamma * q_value_next_adv - q_value)  
            else:  
                # optimal player OR other learning agent if not self_learning  
                if not self_learning or test:  
                    # choosing next action using the optimal policy  
                    move = player_adv.act(heaps)  
                    heaps, end, winner = env.step(move)  
                if not test:  
                    if self_learning:
```

Annexe 2 : code python

```
17 self_learning:
    # q-learning agent 2 choosing next action using q-values
    state_idx_adv = heaps_to_idx(heaps)
    move,q_value_adv,action_adv= player_adv.act_q(heaps, q_table[state_idx_adv], greedy= False)
    heaps, end, winner = env.step(move)

    # q-learning agent 1 choosing next potential greedy-action using q-values, without actually taking it, then updating the q-table
    next_state_ind = heaps_to_idx(heaps)
    _,q_value_next,_= player_q.act_q(heaps, q_table[next_state_ind], greedy= True)
    if not end and env.num_step>1:
        q_table[state_idx][action] += eta * (reward + gamma * q_value_next - q_value)

if end:
    if winner== player_q.player:
        reward = 1
    else:
        reward = -1
    if not test:
        # q-learning agent 1 updating q-table with final reward value
        q_table[state_idx][action] += eta * (reward + gamma * q_table[0][0] - q_value)
        if self_learning:
            # q-learning agent 2 updating q-table with final reward value
            q_table[state_idx_adv][action_adv] += eta * (-reward + gamma * q_table[0][0] - q_value_adv)
    break
return reward
```

Annexe 2 : code python

```
def q_learning(env = None, q_table=None, num_games= 20000, eta=0.1, gamma=0.99, eps_q=0.2, eps_adv= 0.5, initial_table_value=0, decreasing_eps= False):  
  
    if env is None:  
        env = NimEnv()  
    if q_table is None:  
        q_table = create_q_table(NUM_HEAPS,MAX_STICKS,initial_table_value)  
  
    # initializing metrics  
    rewards = [] # -1 or 1, stored at the end of each game  
    rewards_mean=[] # mean of every x rewards (here 250) stored after the x_th game  
    m_opt=[]  
    m_rand=[]  
    start = time.time()  
    Turn = 0  
    # both the agent and the adversary are OptimalPlayer instances but the agent is calling act_q whereas adv is calling act  
    # if self_learning = True, both are calling act_q  
    for i in tqdm(range(num_games), disable=(not verbose)):  
        if decreasing_eps:  
            eps = max(eps_min,eps_max*(1-i/n_star))  
            player_q = OptimalPlayer(epsilon=eps, player=Turn)  
        else:  
            eps = eps_q  
            player_q = OptimalPlayer(epsilon=eps, player=Turn)  
        if self_learning:  
            # at the beginning of every game, reinstantiate the optimal player (especially if eps changes, turn changes anyway)  
            player_adv = OptimalPlayer(epsilon=eps, player= 1-Turn)  
        else:  
            player_adv = OptimalPlayer(epsilon=eps_adv, player=1-Turn)  
        rewards.append(play_game(env, q_table, player_q, player_adv, eta, gamma, False, self_learning=self_learning))  
        if i%METRICS_FREQUENCY==0 and i!=0:  
            rewards_mean.append(np.asarray(rewards).mean())  
            rewards = []  
            if test: # plays against optimal player (OPT(0)) and random player (OPT(1))  
                for eps_opt in [0,1]:
```

Annexe 2 : code python

```
def q_learning(env = None, q_table=None, num_games= 20000, eta=0.1, gamma=0.99, eps_q=0.2, eps_adv= 0.5,
               initial_table_value=0, decreasing_eps= False, eps_min=0.1, eps_max=0.8, n_star=1, test=False, verbose= False, self_learning=False):

    if env is None:
        env = NimEnv()
    if q_table is None:
        q_table = create_q_table(NUM_HEAPS,MAX_STICKS,initial_table_value)

    # initializing metrics
    rewards = [] # -1 or 1, stored at the end of each game
    rewards_mean=[] # mean of every x rewards (here 250) stored after the x_th game
    m_opt=[]
    m_rand=[]
    start = time.time()
    Turn = 0
    # both the agent and the adversary are OptimalPlayer instances but the agent is calling act_q whereas adv is calling act
    # if self_learning = True, both are calling act_q
    for i in tqdm(range(num_games), disable=(not verbose)):
        if decreasing_eps:
            eps = max(eps_min,eps_max*(1-i/n_star))
            player_q = OptimalPlayer(epsilon=eps, player=Turn)
        else:
            eps = eps_q
            player_q = OptimalPlayer(epsilon=eps, player=Turn)
        if self_learning:
            # at the beginning of every game, reinstantiate the optimal player (especially if eps changes, turn changes anyway)
            player_adv = OptimalPlayer(epsilon=eps, player=1-Turn)
        else:
            player_adv = OptimalPlayer(epsilon=eps_adv, player=1-Turn)
        rewards.append(play_game(env, q_table, player_q, player_adv, eta, gamma, False, self_learning=self_learning))
        if 1%METRICS_FREQUENCY==0 and i!=0:
            rewards_mean.append(np.asarray(rewards).mean())
            rewards = []
        if test: # plays against optimal player (OPT(0)) and random player (OPT(1))
```

Annexe 2 : code python

```
rewards = []
if test: # plays against optimal player (OPT(0)) and random player (OPT(1))
    for eps_opt in [0,1]:
        won_count=0
        for ii in range(500):
            heaps, __, __ = env.observe()
            player_q = OptimalPlayer(epsilon=0, player=Turn)
            player_opt = OptimalPlayer(epsilon=eps_opt, player=1-Turn)
            if 1 == play_game(env, q_table, player_q, player_opt, eta, gamma, True):
                won_count +=1
            Turn = 1 - Turn
        m=(won_count-(500-won_count))/500
        if eps_opt==0:
            m_opt.append(m)
        else:
            m_rand.append(m)
    Turn = 1 - Turn
metrics = []
metrics.append(rewards_mean)
if test:
    # list of three lists, agent, optimal player and random player
    metrics.append(m_opt)
    metrics.append(m_rand)
elapsed_time = time.time() - start
print(f"Elapsed time : {elapsed_time}")
return metrics, q_table
```

Annexe 2 : code python

```
env = NimEnv()
metrics, _ = q_learning(env, num_games=20000, eps_q=0.1, gamma= 0.99, verbose=False)
x = np.arange(1, len(metrics[0])+1)*METRICS_FREQUENCY
training_results.append([x, metrics])
x = training_results[0][0]
metrics = training_results[0][1].copy()
fig = plt.plot(x=x, y=metrics, title=f'Average reward over time of RL agent with policy epsilon={0.1}')
fig.update_layout(xaxis_title='Games played', yaxis_title='Average reward')
fig.update_layout(showlegend=False)
fig.update_layout(width=600)
fig
```